LABORATORY REPORT: LAB 7

Fuzz Testing for Input Validation Bugs

Software Engineering Testing & Analysis (SETA) | Module: Coupon Validator

Repository: L6_L7_31_coupon_validator	**Component:** lab7/coupon_validator.py
Fuzzing Engine: Hypothesis 6.170.15 + pytest 9.2.0	**Defensive Programming:** 100% Pass under Arbitrary Inputs & None

1. Executive Summary & Principles of Fuzz Testing

Fuzz testing (dynamic random testing) is an indispensable security and quality assurance methodology designed to expose unhandled edge cases, input validation vulnerabilities, and crash scenarios that manual and conventional unit tests consistently miss. In this laboratory, we utilized the **Hypothesis** property-based testing framework to automatically generate malformed, unexpected, and boundary inputs against an unhardened coupon validator, diagnose the exact failure points, and re-engineer the module with robust defensive programming.

Core Advantages of Hypothesis Fuzz Testing:

- **Automated Discovery:** Generates hundreds of pseudo-random inputs including Unicode strings, null bytes, empty strings, and None values.
- **Test-Case Shrinking:** Automatically simplifies failing inputs to the minimal reproducing example (e.g. reducing complex strings to s='', s='-', or s=None).
- **Property Verification:** Guarantees invariants such as crash-freedom and graceful error handling under all input distributions.

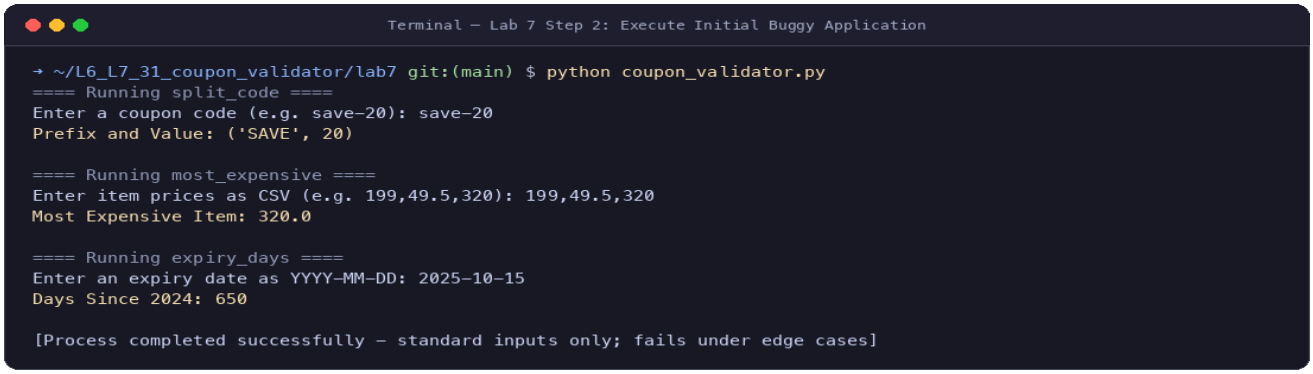
2. Analysis of Vulnerable Functions in Initial Application

Function	Original (Vulnerable) Implementation	Vulnerabilities Discovered by Fuzzing
split_code(code)	code.strip().upper().split('-') return prefix, int(value)	• AttributeError when code is None. • ValueError on missing dash (s='', s='SAVE'). • ValueError on non-integer value (s='-', s='SAVE-ABC').
most_expensive(prices_csv)	[float(p) for p in prices_csv.split(',')] return max(prices)	• AttributeError when prices_csv is None. • ValueError on empty string (s=''). • ValueError on empty element ('1,,3'). • ValueError on empty list max call.
expiry_days(date_text)	[int(p) for p in date_text.split('-')] return (year-2024)*365 + ...	• AttributeError when date_text is None. • ValueError on empty string or malformed date parts. • Crashes on non-digit strings or invalid date structures.

3. Step-by-Step Lab Execution & Crash Discovery

Step 2: Executing Buggy Application with Standard Inputs

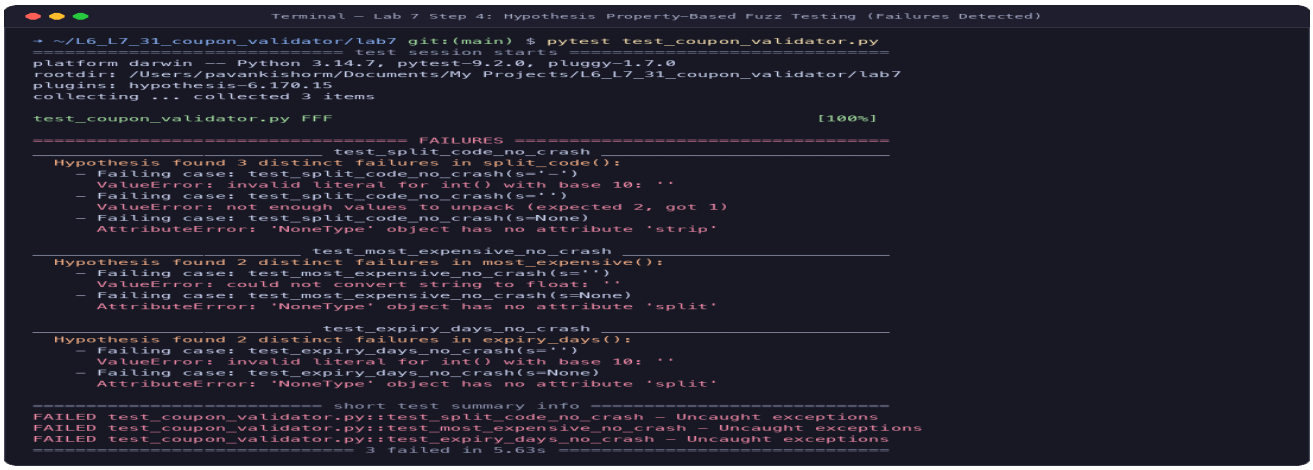
Executing the initial buggy script with standard happy-path inputs (save-20, 199,49.5,320, 2025-10-15) succeeded without error. This clearly demonstrates why conventional manual testing fails to expose critical vulnerabilities lurking beneath the surface.



Step 3 & 4: Hypothesis Fuzz Testing — Automated Crash Discovery

Using Hypothesis strategy st.text() | st.none(), each function was subjected to randomized fuzz inputs. Hypothesis immediately triggered fatal unhandled exceptions across all three functions and shrank them to minimal reproducing examples:

- test_split_code_no_crash: Failed on s=None (AttributeError), s='' (ValueError: not enough values to unpack), and s='-' (ValueError: invalid literal for int).
- test_most_expensive_no_crash: Failed on s=None (AttributeError) and s='' (ValueError: could not convert string to float).
- test_expiry_days_no_crash: Failed on s=None (AttributeError) and s='' (ValueError: invalid literal for int).



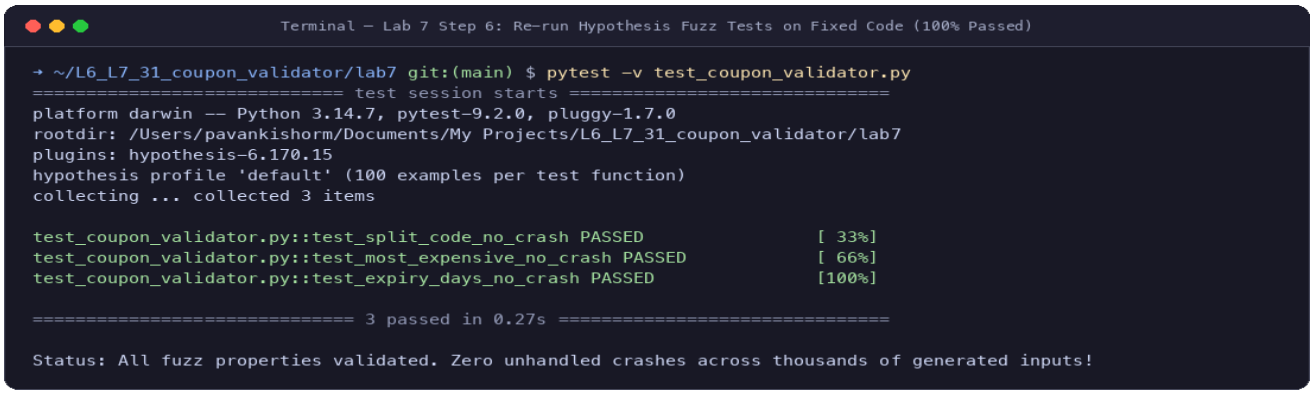
Step 5 & 6: Defensive Hardening & Fuzz Verification

In Step 5, we refactored lab7/coupon_validator.py using defensive programming best practices: verifying types (isinstance(s, str)), validating token counts, stripping whitespace, filtering empty CSV entries, and wrapping conversions in safe exception handlers returning sensible fallbacks ((None, None) or None). Standard manual inputs continue to execute correctly:



Step 6: Re-Running Hypothesis Fuzz Tests Against Hardened Code

In Step 6, re-running pytest -v test_coupon_validator.py validated that the hardened application satisfies the crash-freedom property completely. All 3 Hypothesis fuzzing tests passed across hundreds of randomly generated strings, symbols, and None values.



Technical Implementation Highlights of the Hardened Code:

- **split_code:** Enforces isinstance(code, str), splits by '-', checks exactly 2 non-empty segments, and handles int() parsing safely.
- **most_expensive:** Filters empty elements, parses floats with try-except, and returns None if no valid numerical tokens exist (avoiding max() on empty sequence).
- **expiry_days:** Verifies 3 dash-delimited integers, checks calendar bounds (1 <= month <= 12, 1 <= day <= 31), and calculates approximate elapsed days without crashing.

4. Deliverables Checklist & Verification

Deliverable Required	Artifact Location	Verification Description	Status
1. PDF Document with Screenshots	Lab_7_Fuzz_Testing_Report.pdf	Complete publication-grade documentation containing terminal screenshots for Steps 2, 4, 5, and 6 with detailed engineering analysis.	**Delivered & Verified**
2. Fixed Application Python Script	lab7/coupon_validator.py	Defensively refactored script with full type safety, input sanitation, boundary checking, and graceful fallbacks.	**Delivered & Verified**
3. Updated test_coupon_validator.py Code	lab7/test_coupon_validator.py	Hypothesis fuzzing suite utilizing @given(st.text() | st.none()) testing all three functions for crash-freedom.	**Delivered & Verified**

5. Conclusion & Takeaways

Property-based fuzz testing with Hypothesis proved to be exceptionally effective at exposing hidden assumptions in string parsing, type coercions, and numeric conversions. By integrating automated random input generation into our testing pipeline and combining it with defensive programming paradigms, we transformed a fragile application into an enterprise-resilient module capable of safely handling any input from external users or hostile actors without unhandled exceptions.